Garantie de build, procédure de mise à jour et politique de langue

Produit : pili-pos-android 1.0.0 (posSwVersion pili-pos-android/1.0.0) **Plateforme :** Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Date :** À effectuer (EV-18 : date de dépôt) **Sections couvertes :** §II.D Garantie du produit, §II.E Mises à jour, §VI Règles de langue du manuel de certification.

---

1. Garantie d'identité machine-test <-> déploiement client (§II.D)

Question du manuel : "Quelles sont les garanties que vous avez mis en place afin de s'assurer que le logiciel de la machine-test et celui du SCE installé chez le client sont identiques ?"

Réponse :

La correspondance exacte entre le build de certification soumis au SPF Finances et le build déployé chez chaque client est garantie par quatre mécanismes complémentaires :

1.1 Build reproductible

Le dépôt source contient : - `Cargo.lock` (racine) et `android/rust/Cargo.lock` commités - figent les versions exactes de toutes les dépendances Rust transitives ; le script de build compare leur SHA-256 avant et après la construction et échoue si l'un des deux a changé. - `rust-toolchain.toml` épingle la version exacte du compilateur Rust (canal `1.99.0`). - `android/gradle/wrapper/gradle-wrapper.properties` épingle Gradle 8.11.1 par son SHA-256 ; `android/gradle/verification-metadata.xml` fige le SHA-256 de chaque dépendance Gradle. - L'interface est écrite en Kotlin : aucune dépendance Node.

L'APK non signé de la build cert est produit dans un conteneur Debian 12 `linux/amd64` (`android/docker/Dockerfile`) par `android/docker/build_in_container.sh`, qui exécute `:app:assembleRelease` hors réseau, les dépendances ayant été préchargées une fois et vérifiées par leurs empreintes (Gradle appelle cargo-ndk pour compiler le moteur en `libpos_ffi.so`) :

```
scripts/android/check_repro.sh   # deux builds du même commit, comparées octet par octet
```

Le conteneur fixe l'environnement de compilation : - l'image de base est désignée par son empreinte (`debian:bookworm-slim@sha256:…`), les paquets apt par une date de snapshot, et chaque téléchargement (JDK 17, outils de ligne de commande Android, Rust 1.99.0, cargo-ndk 4.1.2, NDK r27b, build-tools 35.0.0) est vérifié par une empreinte ; - l'empreinte de l'image de build, relevée à la date de la build de record : À effectuer (EV-05 : empreinte de l'image et SHA-1 des deux APK non signés identiques) ; - la date des fichiers est fixée au moment du commit (`SOURCE_DATE_EPOCH`) et les chemins de l'hôte (dépôt, registre Cargo) sont remplacés dans le code compilé par des noms fixes (`--remap-path-prefix`) ; - la clé de signature n'entre jamais dans le conteneur : le conteneur produit un APK non signé, la signature est faite ensuite (§1.2).

Deux builds du même commit produisent donc le même APK non signé, et la même bibliothèque `libpos_ffi.so`. Le producteur exécute `scripts/android/check_repro.sh` sur le commit tagué avant chaque déclaration (`docs/android/REPRO.md`) : deux builds complètes dans deux conteneurs sans réseau, comparaison des APK, des bibliothèques natives et de l'inventaire des entrées. Le rapport de cette exécution sur le tag cert est joint au dossier : À effectuer (EV-05 : rapport de `check_repro.sh` au commit de release).

1.2 Empreinte SHA-1 de l'APK

L'APK non signé de §1.1 est signé hors du conteneur, avec l'outil `apksigner` épinglé (build-tools 35.0.0), une clé RSA, les schémas v2 et v3 seulement (`--min-sdk-version 33`, v1 et v4 désactivés). La signature RSA est déterministe : signer deux fois le même APK non signé avec la même clé donne le même fichier, donc le même SHA-1. Une clé de signature à courbe elliptique est refusée par le script de déclaration pour cette raison.

Procédure, dans l'ordre : 1. **Vérification de l'APK non signé** (`scripts/android/check_apk.sh`) : alignement des pages sur 16 Ko, une seule architecture (`arm64-v8a`), bibliothèques `libpos_ffi.so` et `libjni-dispatch.so` présentes, non compressées et alignées, `libpos_ffi.so` étant en outre allégée des symboles

de débogage. Le moteur est compilé avec la fonction `cert` (HubRise et facturation B2B inclus, aucune fonction Cargo supplémentaire, `android/rust/pos-ffi/Cargo.toml`). 2. **Signature** de l'APK avec la clé de release RSA, puis `apksigner verify --print-certs` (un seul signataire, schéma v3 vrai, v1 faux). 3. **SHA-1 de l'APK signé de record** : À effectuer (EV-01 : SHA-1 et SHA-256 de l'APK signé `pili-pos-android 1.0.0`). C'est la valeur déclarée au formulaire §I.D et au tableau §VII.F. 4. **SHA-1 de la bibliothèque du moteur** lib/arm64-v8a/libpos_ffi.so extraite de cet APK : À effectuer (EV-02 : SHA-1 de `libpos_ffi.so`). 5. **Empreinte SHA-256 du certificat de signature** (celui de la clé de release) : À effectuer (EV-03 : SHA-256 du certificat de signature). Android n'installe une mise à jour que si elle porte la même clé : cette empreinte est aussi copiée dans le descripteur de mise à jour déclaré (§2.1). 6. **Version du moteur** : le moteur est celui de l'étiquette de gel android-v1.0.0-rc3 (§1.4), dont le commit est À effectuer (EV-04 : commit de l'étiquette de gel android-v1.0.0-rc3, puis de l'étiquette finale android-v1.0.0).

La signature se fait après la vérification de reproductibilité (§1.1), et le script de déclaration refuse tout APK dont le SHA-1 signé diffère de la valeur acceptée par le SPF Finances.

Vérification sur une caisse. L'application calcule elle-même les empreintes de l'APK installé et les affiche : écran "À propos", sections "Version" et "Intégrité" : - "SHA-1 de base.apk" et "SHA-256 de base.apk" ; - "SHA-1 de libpos_ffi.so" ; - "SHA-256 du certificat de signature" ; - version de l'application, code de version, "Version logicielle du moteur" (pili-pos-android/1.0.0) et identifiant de l'appareil (deviceId).

Ces valeurs doivent être égales à celles de EV-01, EV-02 et EV-03. Les mêmes valeurs figurent dans le fichier d'identité (`identity.json`) écrit à chaque export SPF sur clé USB ou carte SD. Sur une caisse de production le débogage (adb) est coupé : l'empreinte se lit donc dans l'application, jamais par un outil branché.

Vérification indépendante, sans la caisse :

```
# 1. L'artefact public déclaré (ghcr.io/brunomoyse/moyun-pos, étiquette declared-android)
#    contient declared.json, sa signature minisign et l'APK
shasum -a 1 pili-pos-android-1.0.0.apk            # doit correspondre à EV-01
unzip -p pili-pos-android-1.0.0.apk lib/arm64-v8a/libpos_ffi.so | shasum -a 1   # EV-02
apksigner verify --print-certs pili-pos-android-1.0.0.apk   # SHA-256 du certificat : EV-03
# 2. Reconstruction depuis le tag android-v1.0.0
scripts/android/check_repro.sh
```

La reconstruction donne l'APK non signé, qui n'a pas la signature du producteur : son inventaire d'entrées (`scripts/android/apk_entry_diff.py`) se compare à celui de l'APK signé, car il ignore le bloc de signature.

1.3 Environnement verrouillé

Le premier déploiement se fait par le producteur, qui installe l'APK sur le terminal et le met sous propriétaire d'appareil (*MANUEL_INSTALLATION_FR.md*) ; les mises à jour remplacent l'APK par le seul outil de mise à jour déclaré (cf. §2.1). **Le programme de mise à jour de l'application est le seul chemin d'arrivée de l'APK fiscal** : jamais par adb (coupé en production), jamais par une source inconnue, jamais pendant une session d'assistance à distance (il n'en existe pas sur Android, cf. §2.2). Sur le terminal avec services Google, tel que construit, la boutique d'applications et les applications Google destinées à l'utilisateur sont masquées, les installations sont bloquées (`DISALLOW_INSTALL_APPS`, sources inconnues) sauf la session de l'outil de mise à jour (`PLATEFORME_ANDROID_FR.md` §3.3 et §3.7). Mesuré sur le profil émulé le 15.10.2026 (EV-13, `evidence/gms/EV13_2026-10-15.md`) : onze paquets Google présents et masqués, mise à jour déclarée installée sous la restriction. Comportement de Play Protect sur un vrai terminal : À effectuer (EV-15 : Play Protect sur le D3 Pro réel).

Le terminal est configuré lors de la mise en service pour constituer un environnement verrouillé (détail : `PLATEFORME_ANDROID_FR.md` §3) :

- L'application est propriétaire de l'appareil (device owner) et tourne seule en mode Lock Task : l'écran ne montre que pili-pos-android, aucun terminal ni lanceur accessible à l'opérateur.
- Restrictions posées par le propriétaire de l'appareil : installation d'applications interdite (levée par l'outil de mise à jour pour sa seule session) et installation depuis des sources inconnues interdite, désinstallation de l'application interdite, mode sans échec et réinitialisation d'usine interdits, changement de la date et de l'heure interdit (fuseau `Europe/Brussels`), ajout de comptes et d'utilisateurs interdits.

- Débogage USB coupé sur une caisse de production : dernière étape de la mise en service, appliquée dès qu'un paquet de mise en service est installé.
- Aucun serveur n'écoute sur le terminal.
- La base de données, les sauvegardes, les journaux et la configuration sont dans le stockage privé de l'application (`files/engine/`, `PLATEFORME_ANDROID_FR.md` §5), inaccessible aux autres applications. Le chiffrement du stockage et l'état du démarrage vérifié (Verified Boot) du terminal réel : À effectuer (EV-15 : chiffrement du stockage et Verified Boot du D3 Pro).

Une fois la mise en service effectuée, toute modification de l'APK passe obligatoirement par l'outil de mise à jour (§2.1), qui refuse tout APK dont l'empreinte ou le certificat ne correspondent pas à la déclaration.

1.4 Tags git

Le commit du build de certification est tagué `android-v1.0.0` (le nom du tag est la `versionName` de l'APK, vérifiée par le script de déclaration) ; le code de version est 10000. Le moteur fiscal est celui de l'étiquette de gel android-v1.0.0-rc3 (puis de l'étiquette finale `android-v1.0.0`), dont l'absence de modification est contrôlée par `scripts/android/check_android_frozen.sh` : À effectuer (EV-04 : commit de l'étiquette de gel android-v1.0.0-rc3, puis de l'étiquette finale android-v1.0.0). Le SHA git du tag est consigné dans le rapport de build et permet à tout moment de reproduire le build à l'identique à partir du dépôt source.

Toute mise à jour après la certification (correction de bug, nouvelle fonctionnalité) passe par : - Un nouveau tag `android-vX.Y.Z+1` et un code de version strictement supérieur. - Une nouvelle build cert reproductible (et donc nouveau SHA-1). - Une nouvelle demande de certification SPF Finances pour la mise à jour (procédure §II.E ci-dessous + Article 109 du Manuel). Tout changement, du moteur ou de l'application, donne une nouvelle version de `pili-pos-android` (*CADRE_DEMANDE_FR.md* §6).

---

2. Procédure de mise à jour (§II.E)

Question du manuel : "Pour chaque modification sur le POS, une demande de certification de mise à jour est nécessaire. Ces mises à jour sont testées sur la machine de test. Comment implémentez-vous les mises à jour sur la machine de test ?"

Réponse :

2.1 Mises à jour entre cycles de certification

Aucune mise à jour automatique en arrière-plan n'est implémentée : l'outil ne vérifie rien tant que le Gérant ne le démarre pas, et aucune installation n'est planifiée ni déclenchée à distance. C'est une décision explicite de scellement de l'appliance fiscale : un POS certifié ne doit pas pouvoir voir son code changé sans une nouvelle validation SPF Finances. L'outil n'installe que ce qui a été déclaré (étiquette `declared-android`) ; rien d'autre.

La procédure de mise à jour est la suivante :

1. **Côté producteur :**
   - Implémentation des modifications dans une branche dédiée.
   - Tests automatisés (moteur, adaptateur `android/rust`, application Kotlin) passent à 100% : À effectuer (EV-17 : comptes de tests au commit de release).
   - Anti-régression : `scripts/android/check_apk.sh` sur l'APK non signé (§1.2).
   - Nouveau tag `android-vX.Y.Z+1`, puis `scripts/android/check_repro.sh` sur le commit tagué : deux APK non signés identiques.
   - Signature de l'APK avec la clé de release, calcul du SHA-1 de l'APK signé.
   - Soumission d'une nouvelle demande de certification (mise à jour) au SPF Finances.
   - Après acceptation du SHA-1 par le SPF Finances seulement : `scripts/release/declare_android.sh` signe, vérifie que le SHA-1 obtenu est la valeur acceptée, puis publie les trois fichiers de la déclaration (`declared.json`, sa signature minisign et l'APK). Rien n'est publié si une vérification échoue.

2. **Sur la machine de test SPF (pendant le cycle de mise à jour) :**
    - Le producteur remet les trois fichiers de la déclaration sur clé USB, sur place (cf. §2.2).
    - Le Gérant ouvre “Config” > “Mise à jour” > “Depuis une clé USB” et choisit le dossier de la clé. L’outil applique les mêmes contrôles qu’en ligne (ci-dessous).
    - Tests de validation effectués par l’agent SPF.
3. **Côté client (après acceptation SPF de la mise à jour) :**
    - Mise à jour de l’APK par le Gérant, sur site, avec la même procédure : “Vérifier en ligne” (téléchargement anonyme de l’artefact public déclaré) ou “Depuis une clé USB”.
    - Sauvegarde préalable de la base `pos.db` : l’outil en écrit une avant d’installer l’APK (la caisse en écrit aussi une après chaque rapport Z et conserve les 7 plus récentes : `4ff25b110c975d497f548538a08d81387b504b` (commit « feat(engine): keep only the 7 newest database backups » du 2026-10-10, contenu dans l’étiquette de gel android-v1.0.0-rc3)).
    - Vérification post-update du SHA-1 : l’écran “Mise à jour” et l’écran “À propos” affichent le SHA-1 de l’APK installé ; l’application l’écrit aussi, avec le SHA-1 déclaré, dans le journal d’audit.

Conditions de démarrage. La mise à jour est refusée, avec le motif à l’écran, tant que : - l’opérateur connecté n’est pas un Gérant ; - une commande ou une vente à emporter est ouverte ; - un rapport Z est dû, ou des opérations fiscales (hors pointages) ont eu lieu depuis le dernier rapport Z ; - le moteur de la caisse n’est pas prêt, ou ne s’arrête pas à temps avant l’installation (la caisse continue alors de fonctionner).

Contrôles de la déclaration, dans cet ordre, au premier refus tout s’arrête et le fichier téléchargé est effacé : 1. une clé publique de déclaration est installée dans l’application (sinon refus) ; 2. la signature minisign de `declared.json` est valide pour cette clé ; 3. le descripteur est complet (plate-forme `android`, version, code de version, empreintes SHA-1 et SHA-256, empreinte du certificat) ; 4. l’identifiant de l’application est celui de l’application installée ; 5. le code de version est **strictement supérieur** à celui de l’application installée ; 6. le SHA-256 puis le SHA-1 de l’APK téléchargé sont ceux du descripteur ; 7. l’APK a le même identifiant, le même code de version et le même nom de version que le descripteur, et il est signé par le **même certificat** que l’application installée, celui du descripteur.

Installation. Elle passe par une session du programme d’installation d’Android (PackageInstaller), ouverte par le propriétaire de l’appareil : c’est le seul chemin d’installation de l’APK fiscal sur ce terminal. Si le système exige une confirmation à l’écran, la mise à jour est refusée. À la fin, l’application redémarre seule sur l’écran de connexion. Chaque tentative, refusée ou réussie, est écrite dans le journal d’audit de la caisse (type `update`).

Pas de retour arrière. L’outil ne propose aucun retour à une version antérieure : Android refuse un APK de code de version inférieur sans désinstallation préalable, et la désinstallation supprime la base fiscale. Une version défectueuse est corrigée par une nouvelle version déclarée.

Correctifs urgents. La procédure du SPF Finances (Introduction à la certification, 2026-09-03, §4) prévoit : « Les mises à jour ayant un impact sur le volet SCE ne peuvent être installées chez les clients qu’après avoir été certifiées par nos soins. Les corrections urgentes de bugs ou les solutions à des blocages chez des clients constituent une exception à cette règle, mais doivent tout de même nous être signalées. » pili-pos-android s’y conforme : un correctif urgent peut être installé chez un client avant la nouvelle certification, à condition d’être signalé au SPF Finances, et il figure dans la demande de mise à jour suivante. Il suit le même chemin que toute mise à jour : il est déclaré, puis installé par l’outil de mise à jour. Dans la base, les versions de migrations 12 à 19 sont réservées à ces correctifs certifiés (`crates/pos-db/src/migrate.rs`).

2.2 Accès à distance

Le manuel recommande l’usage d’une solution d’accès à distance (TeamViewer, AnyDesk, RustDesk, VNC, Zoho Assist, Splashtop) pour faciliter les mises à jour et l’assistance technique.

Position de pili-pos-android : aucune solution d’accès à distance, ni permanente ni à la demande, n’existe sur Android : ni sur la machine de test livrée à l’Intake, ni sur la production client. C’est une décision délibérée :

- **Raison principale :** ouvrir un canal d’administration distante sur un terminal de caisse fiscale augmente la surface d’attaque sans bénéfice opérationnel suffisant. Un attaquant qui compromet l’accès distant peut potentiellement contourner les protections fiscales.

- **Raison secondaire :** la spec SCE 2.0 demande un appliance scellé. Un agent permanent ou un client d'assistance contredit l'esprit du scellement.

Conséquences dans le produit : le moteur est ouvert sans programme d'assistance ; la commande de session d'assistance renvoie une erreur codée "indisponible" et l'interface ne présente aucune entrée d'assistance à distance. Aucun port n'écoute sur le terminal ; les seules connexions sont sortantes : FDM, HubRise (si activé), envoi des tickets par courriel, export nocturne (si configuré) et, à la demande du Gérant, le téléchargement de la mise à jour déclarée. Le débogage USB est coupé en production.

Assistance sur place : pour l'assistance et le diagnostic d'un problème reporté par le client (jamais pour installer un APK hors de l'outil de mise à jour), le producteur se déplace. Il dispose des écrans de la caisse sous PIN de Gérant, du paquet de mise en service signé (configuration, jeton et certificats du FDM) et de l'export des journaux et des sauvegardes vers une clé USB ou une carte SD (*PLATEFORME_ANDROID_FR.md* §5).

Mises à jour du système Android. Elles sont distinctes de celles de l'application et ne changent jamais l'APK déclaré ni la base de données. Comme construit, le propriétaire de l'appareil applique une politique de mise à jour du système à fenêtre horaire : les mises à jour ne s'installent que de 03:00 à 05:00, hors des heures de service. Politique relevée sur le profil émulé avec services Google (15.10.2026, EV-13, `evidence/gms/EV13_2026-10-15.md`) : fenêtre de 03:00 à 05:00, aucun gel. Déclenchement réel d'une mise à jour du système dans la fenêtre : À effectuer (EV-15 : mise à jour du système sur le D3 Pro réel).

Au rendez-vous d'admission SPF Finances :

- Accès physique direct de l'agent à l'écran de la machine de test.
- Données par l'export SPF sur clé USB ou carte SD (*FEUILLE_ACQUISITION_DONNEES_FR.md*).
- Aucun besoin d'accès à distance pendant l'Intake.

---

3. Politique de langue des tickets (§VI)

Question du manuel : règles de langue selon §VI - le ticket doit être disponible en français, néerlandais, allemand selon le besoin (anglais en zones touristiques).

Réponse :

3.1 Langues supportées par le code source

Le moteur de `pili-pos-android` supporte le rendu multilingue des tickets via deux mécanismes complémentaires :

- **Libellés produits** : table `translations` polymorphe (cf. *SCHEMA_BD_FR.md*) - couvre catégories, articles, modificateurs. Les lignes du ticket portent le libellé dénormalisé dans la langue active au moment de la vente.
- **Mentions fiscales imprimées** : jeux de libellés FR/NL/DE embarqués dans le code (`crates/pos-fiscal/src/ticket.rs` : `ReceiptLanguage`, `ticket_type_label`, `invalid_disclaimer` ; `crates/pos-print/src/labels.rs` : libellés fixes Date/Ticket/TOTAL/TVA du reçu et du rapport, noms d'événement N/P/T/C/I/F/S/R, noms de moyen de paiement). Les rapports X et Z, le rapport utilisateur (M182/M183/M184) et les noms de moyen de paiement imprimés (Especes/Contant/Bar, Carte/Kaart/Karte) suivent la même langue que le reçu : ce n'est pas propre au ticket de vente. La langue est déterminée par la clé `language` du fichier de configuration `pos.json`, que le moteur lit dans le paquet de mise en service signé installé sur la caisse, mappée sur une langue nationale : `fr`/`nl`/`de` directement, toute autre valeur (en, zh) retombe sur le FR.

Langues des mentions fiscales supportées par le code : - `fr` (français) - **complète** (libellés issus de la description détaillée POS v2.2 et du manuel §VI) - `nl` (néerlandais) - **complète** (libellés issus des circulaires GKS) - `de` (allemand) - **complète** (traduction de style officiel, en ASCII ; les libellés allemands ne sont pas tirés d'une version allemande de la spécification, qui ne fait pas partie des documents de référence)

Langues d'affichage de l'interface (indépendantes du ticket) : `fr`, `en`, `zh`, au choix de l'opérateur sur l'écran de connexion.

Documents non fiscaux des fonctions du propriétaire : le bon cadeau et son solde sont imprimés en français, néerlandais ou allemand (langue de l'établissement ; toute autre valeur retombe sur le français) ; la fiche d'allergènes en français, néerlandais, anglais ou allemand ; les bons de commande par temps (« Direct », « Entrées », « Plats », « Desserts ») en français, néerlandais ou allemand. Les messages d'alerte du propriétaire (hors ticket) existent en français, en chinois et en anglais, selon la langue réglée par le Gérant. Le nom du moyen de paiement « Bon cadeau » suit la langue du ticket : « Bon cadeau », « Cadeaubon », « Geschenkgutschein ».

3.2 Règle appliquée : langue de la région linguistique de l'établissement

Le manuel (§VI, p. 99) dispose : *« En Belgique, le ticket doit être émis en français, en néerlandais ou en allemand, selon la langue officielle de la région linguistique où il est émis. Dans les lieux considérés comme des zones touristiques, le ticket peut également être émis en anglais. »*

La langue des mentions fiscales est donc fixée **par installation**, lors de la mise en service, d'après l'adresse de l'établissement où le POS est exploité (et non d'après le siège du producteur ni la langue de la clientèle) :

Région linguistique de l'établissement	Valeur de `pos.json::language`
Région de langue française	`fr`
Région de langue néerlandaise	`nl`
Région de langue allemande	`de`
Région bilingue de Bruxelles-Capitale	`fr` ou `nl` (les deux langues sont officielles)

L'anglais (zones touristiques) n'est pas proposé comme langue de ticket : toute valeur autre que `fr`/`nl`/`de` retombe sur le FR (cf. §3.1).

Le paramètre fait partie de la checklist de mise en service (`MANUEL_INSTALLATION_FR.md`) et figure sur chaque installation déclarée dans `LISTE_DISTRIBUTEURS_FR.md`.

Exemple : un établissement situé à 4000 Liège se trouve en **région de langue française** → `language = "fr"`, tickets imprimés en français.

Changement de langue. Dans la version 1.0.0, la langue des mentions fiscales est fixée à la mise en service, par la clé `language` du fichier `pos.json` du paquet de mise en service signé. La caisse en service n'offre aucun écran pour la modifier : l'écran d'import du paquet n'est présenté que sur une caisse non mise en service ou dont l'identifiant d'appareil ne correspond pas, et le script de base initiale (`seed.sql`) est ignoré quand la base existe déjà. Un établissement qui change de région linguistique est donc traité comme une nouvelle mise en service, avec un nouveau paquet signé portant la nouvelle valeur (`fr` / `nl` / `de`) ; la procédure et le sort des données fiscales de l'ancienne installation relèvent du producteur et du SPF Finances (*MANUEL_INSTALLATION_FR.md*), et ne sont pas décrits ici comme une opération de la caisse.

Les trois jeux de mentions FR/NL/DE étant déjà compilés dans l'APK, le choix de la langue ne demande aucune modification de l'APK `pili-pos-android` : c'est de la **configuration** du paquet de mise en service, pas du code.

3.3 Tickets multilingues pour la machine de test SPF

Pour la machine de test livrée à l'Intake, la base initiale (`seed.sql` du paquet de mise en service) sera configurée avec les traductions FR/NL/DE/EN des produits du manuel (Dry Martini, Burger of the Chef, etc.). La machine de test est mise en service avec une seule langue de mentions fiscales, celle de son paquet : l'agent SPF constate cette langue sur les tickets, mais ne la change pas sur la caisse en service dans la version 1.0.0. S'il demande un jeu de tickets dans une autre langue, le producteur prépare une machine de test supplémentaire avec un paquet portant l'autre valeur de `language`.

4. Récapitulatif des engagements

Engagement	Mécanisme	Vérifiabilité
Build cert reproductible	`Cargo.lock` (x2), `rust-toolchain.toml`, image de build par empreinte, Gradle par SHA-256, dépendances Gradle vérifiées	`scripts/android/check_repro.sh` : deux builds, même APK non signé (EV-05)
Build = déploiement client	APK signé RSA (v2 et v3), SHA-1 de record	Écran “À propos” (EV-01, EV-02, EV-03) ; `shasum -a 1` de l’artefact public `declared-android`
Pas de modification post-cert	Propriétaire de l’appareil, installation hors outil de mise à jour interdite, désinstallation interdite, débogage coupé ; Android exige le même certificat pour une mise à jour	SHA-1 de l’APK installé comparé au SHA-1 déclaré ; version pili-pos-android/1.0.0 à l’écran “À propos”
Nouvelle MAJ = nouveau cert	Procédure §II.E + Article 109 du manuel	Chaque tag `android-vX.Y.Z+1` ouvre une nouvelle demande SPF
Pas d’accès distant	Pas de programme d’assistance, pas de port en écoute, débogage coupé	Écrans de la caisse : aucune entrée d’assistance ; scan réseau du terminal réel : À effectuer (EV-15 : scan du D3 Pro réel)
Ticket dans la langue de la région linguistique de l’établissement	`translations` (libellés produits) + `pos.json::language` (mentions fiscales FR/NL/DE)	Inspection visuelle du ticket, paramétrage dans le paquet de mise en service, sans changer l’APK